

Report conversation

How can I implement a health check in Docker Compose for Keycloak 21?

To implement a health check for Keycloak 21 in Docker Compose, you can use the `healthcheck` configuration option available in Docker Compose version 2.1 or higher. The `healthcheck` option allows you to define a command or script that Docker will periodically run to check the health of a container.

Here's an example of how you can implement a health check for Keycloak 21 in Docker Compose:

yaml

 Copy code

```
version: '3.8'
services:
  keycloak:
    image: jboss/keycloak:11.0.0
    ports:
      - 8080:8080
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/auth/realms/master"]
      interval: 30s
      timeout: 10s
      retries: 5
```

In this example:

1. We define a service named `keycloak`.
2. We specify the Keycloak Docker image (`jboss/keycloak:11.0.0` in this case).
3. We map the container's port 8080 to the host's port 8080 to access Keycloak.
4. We define the `healthcheck` configuration block for the `keycloak` service.
5. In the `test` field, we specify the command `["CMD", "curl", "-f", "http://localhost:8080/auth/realms/master"]`. This command uses `curl` to

make an HTTP request to Keycloak's `/auth/realms/master` endpoint. The `-f` flag ensures that `curl` returns a non-zero exit code if the request fails.

6. We set the `interval` to `30s`, indicating that Docker should run the health check every 30 seconds.
7. We set the `timeout` to `10s`, specifying the maximum time Docker waits for the health check command to complete.
8. We set the `retries` to `5`, which means Docker will attempt the health check command up to 5 times before considering the container unhealthy.

With this configuration, Docker will periodically run the `curl` command to check the health of the Keycloak container. If the command fails (returns a non-zero exit code) or exceeds the specified timeout, Docker will consider the container unhealthy.

What if `curl` is not installed?

If `curl` is not installed in the Keycloak container, you can use alternative tools like `wget` or `nc` (netcat) to perform the health check. Here's an example of how you can modify the health check command using `wget`:

yaml

 Copy code

```
version: '3.8'
services:
  keycloak:
    image: jboss/keycloak:11.0.0
    ports:
      - 8080:8080
    healthcheck:
      test: ["CMD-SHELL", "wget --quiet --tries=1 --spider http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5
```

In this example, we use the `wget` command with the `--quiet` option to suppress output, `--tries=1` to attempt a single connection, and `--spider` to check the URL

without downloading the content. If `wget` encounters an error or fails to establish a connection, it will return a non-zero exit code, indicating an unhealthy state.

Alternatively, you can use `nc` (netcat) to perform a basic TCP connection check. Here's an example:

yaml

 Copy code

```
version: '3.8'
services:
  keycloak:
    image: jboss/keycloak:11.0.0
    ports:
      - 8080:8080
    healthcheck:
      test: ["CMD", "nc", "-z", "-w2", "localhost", "8080"]
      interval: 30s
      timeout: 10s
      retries: 5
```

In this case, we use the `nc` command with the `-z` option to scan for open ports, `-w2` to specify a 2-second timeout, and `localhost` and `8080` as the host and port to check. If the connection fails or times out, `nc` will return a non-zero exit code, indicating an unhealthy state.

Choose the approach that works best for your use case based on the tools available in your Keycloak container.